Bounded Steady-State Sidecar Analysis for Offset-Free MPC

1 Purpose

This note documents the analysis-only steady-state workflow used beside the normal offset-free MPC notebook. The closed-loop controller remains unchanged. At each simulation step the sidecar inspects the frozen-disturbance steady-state equations implied by the legacy augmentation and then expands that analysis with input box constraints.

The main practical question is:

> If the bounded steady-state input target $u_s^{bounded}(k)$ often lands on the box limits, does this indicate a bad optimization problem or is it the correct behavior of the chosen formulation?

2 Legacy Augmentation and Frozen Disturbance Target

Let $x_k \in \mathbb{R}^n$ be the physical state, $u_k \in \mathbb{R}^q$ the manipulated input, $y_k \in \mathbb{R}^p$ the measured output, and $d_k \in \mathbb{R}^p$ the output-disturbance state. The legacy augmented model used for offset-free MPC is

$$x_{k+1} = Ax_k + Bu_k, \tag{1}$$
$$d_{k+1} = d_k, \tag{2}$$
$$y_k = Cx_k + d_k. \tag{3}$$

At each closed-loop step the observer provides an augmented estimate

$$\hat{z}_k = \begin{bmatrix} \hat{x}_k \\ \hat{d}_k \end{bmatrix},$$

and the sidecar freezes the disturbance target as

$$d_s(k) = \hat{d}_k.$$

3 Exact Unbounded Steady-State Equations

For the current setpoint $y_{sp}(k)$ define

$$r_k := y_{sp}(k) - \hat{d}_k.$$

The exact steady-state equations are

$$(I - A)x_s(k) - Bu_s(k) = 0, \tag{4}$$
$$Cx_s(k) = r_k. \tag{5}$$

Using the stacked unknown

$$w_s(k) = \begin{bmatrix} x_s(k) \\ u_s(k) \end{bmatrix},$$

the sidecar forms

$$M = \begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix}, \qquad b_k = \begin{bmatrix} 0 \\ r_k \end{bmatrix},$$

and computes the unbounded solution from

$$M w_s(k) = b_k.$$

When $(I - A)$ is invertible, the reduced map

$$G = C(I - A)^{-1} B$$

is also available, so the reduced equation becomes

$$G u_s(k) = r_k.$$

4 Input Box Constraints

The bounded expansion introduces the input box

$$u_{\min} \le u_s(k) \le u_{\max},$$

where $u_{\min}$ and $u_{\max}$ are taken from the same input box used for the MPC action bounds, expressed in the same scaled-deviation coordinates as the sidecar solve.

After computing the exact unbounded solution, the sidecar classifies each step into one of three main cases:

1. `exact_bounded`: the exact solution exists and satisfies the box.
2. `exact_unbounded`: the exact solution exists but violates the box.
3. `exact_unsolved`: the exact solve is not reliable enough to trust directly.

5 Bounded Least-Squares Fallback

If the exact solution violates bounds or cannot be solved reliably, the sidecar solves a box-constrained least-squares problem.

5.1 Reduced Form

When the reduced map G is available, the preferred fallback is

$$\min_{u_s} \quad \frac{1}{2} \| G u_s - r_k \|_2^2 \tag{6}$$

$$\text{subject to} \quad u_{\min} \le u_s \le u_{\max}. \tag{7}$$

After solving for u_s, the state target is recovered from

$$x_s(k) = (I - A)^{-1} B u_s(k).$$

5.2 Full Form

If the reduced form is unavailable or not numerically suitable, the sidecar solves the full box-constrained least-squares problem

$$\min_{x_s,u_s} \quad \frac{1}{2}\left\| \begin{bmatrix} I-A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} - \begin{bmatrix} 0 \\ r_k \end{bmatrix} \right\|_2^2 \tag{8}$$

$$\text{subject to} \quad u_{\min} \le u_s \le u_{\max}. \tag{9}$$

In code this is handled as a bounded least-squares problem using `scipy.optimize.lsq_linear`.

6 Current Solver Choice in the Sidecar Code

The current implementation calls `scipy.optimize.lsq_linear` as

$$\texttt{lsq_linear}(G,\ r_k,\ \texttt{bounds} = (u_{\min}, u_{\max})) \tag{10}$$

for the reduced fallback, and

$$\texttt{lsq_linear}(M,\ b_k,\ \texttt{bounds} = (\ell, u)) \tag{11}$$

for the full fallback, with no explicit `method=` argument. Therefore the SciPy default method is used:

`method='trf'`.

This matters for interpretation:

- the *optimization problem* is the convex bounded least-squares problem written above;
- the *algorithm* used to solve it is currently the trust-region reflective linear least-squares method;
- SciPy also offers `method='bvls'`, which is an active-set bounded-variable least-squares method.

So there are really two separate questions:

1. Is the mathematical optimization problem itself appropriate?
2. Is the chosen numerical algorithm the most interpretable one for a small dense box-constrained target-selection problem?

These questions should not be conflated.

7 Why the Reduced Problem Is a Weighted Projection

Assume the reduced map G is square and invertible, and let the exact unbounded input target be

$$u_s^{exact}(k) = G^{-1} r_k.$$

Then the bounded reduced objective can be rewritten as

$$\frac{1}{2}\|Gu - r_k\|_2^2 = \frac{1}{2}\|G(u - u_s^{exact})\|_2^2 \tag{12}$$

$$= \frac{1}{2}(u - u_s^{exact})^T G^T G (u - u_s^{exact}). \tag{13}$$

Therefore the bounded problem is not an arbitrary heuristic. It is the projection of the exact unbounded target onto the feasible box under the positive-definite metric

$$H = G^T G.$$

This identity is central:

- if u_s^{exact} already lies inside the box, then the bounded and unbounded solutions coincide;
- if u_s^{exact} lies outside the box, then the bounded solution is the closest feasible point to u_s^{exact} in the H-weighted norm.

So if $u_s^{bounded}$ lies on a bound, that is often the expected geometric outcome of the metric projection.

8 KKT Conditions and Active Bounds

For the reduced bounded least-squares problem, define

$$J(u) = \frac{1}{2}\|Gu - r_k\|_2^2.$$

Its gradient is

$$\nabla J(u) = G^T(Gu - r_k).$$

The Karush–Kuhn–Tucker conditions for an optimal bounded solution $u^\star$ are:

$$u_{\min} \le u^\star \le u_{\max}, \tag{14}$$

$$G^T(Gu^\star - r_k) + \lambda^+ - \lambda^- = 0, \tag{15}$$

$$\lambda^+ \ge 0, \qquad \lambda^- \ge 0, \tag{16}$$

$$\lambda_i^+(u_i^\star - u_{\max,i}) = 0, \tag{17}$$

$$\lambda_i^-(u_{\min,i} - u_i^\star) = 0. \tag{18}$$

These conditions explain the behavior at the limits:

- if input i is strictly interior, then the corresponding gradient component must vanish;
- if input i is at its upper bound, a nonzero multiplier λ_i^+ can balance a gradient component that would otherwise push the optimum outside the box;
- if input i is at its lower bound, the same happens with λ_i^-.

Hence a bound-active solution is not evidence of a broken solver. It can simply be the KKT-consistent optimum of the chosen convex problem.

9 Problem Versus Algorithm

9.1 The Mathematical Problem

For the reduced fallback, the sidecar solves

$$\min_{u_s \in [u_{\min}, u_{\max}]} \quad \frac{1}{2}\|Gu_s - r_k\|_2^2. \tag{19}$$

This is a convex quadratic optimization problem. If the solver converges, the returned minimizer is a global optimum of that objective.

9.2 The Numerical Algorithm

The current implementation uses SciPy's default trust-region reflective method. For linear least-squares with bounds this is a legitimate solver choice, but it is not the only one. In SciPy's own documentation:

- `trf` is described as an interior-point-like method;
- `bvls` is described as an active-set bounded-variable least-squares method;
- for `bvls`, the stopping condition is tied directly to KKT satisfaction;
- for `trf`, active-set reporting can be tolerance-dependent because iterates remain strictly feasible.

This suggests the following practical interpretation:

- if the concern is *correctness of the optimum*, the current solver is acceptable because the problem is convex;
- if the concern is *active-bound interpretability*, `method='bvls'` may be more transparent for this small dense analysis problem.

10 What the Current Run Statistics Suggest

For the disturbance run inspected in the sidecar artifacts:

- the exact unbounded steady-state target was inside the box on only about 13.9% of steps;
- about 86.1% of steps required the bounded fallback;
- the average exact-box violation in the scaled coordinates was about 120.1;
- the maximum exact-box violation was about 1234.9.

These are not small edge-of-feasibility corrections. They indicate that the exact steady-state target is often far outside the admissible input box. In that regime, a box-constrained projection or least-squares correction will naturally spend substantial time on the boundary.

Therefore, the dominant explanation for "$u_s^{bounded}$ goes to the limits" in the current run is:

> the exact target is usually far outside the feasible region, so the closest feasible approximation under the current residual objective is frequently located on a box face or corner.

That behavior is consistent with the formulation.

11 When Bound Hitting Is Expected

The bounded solution tends to land on faces or corners of the box in three common situations.

11.1 The Exact Target Is Far Outside the Box

If u_s^{exact} lies well outside the feasible set, then its projection onto the box will typically lie on the nearest face or corner in the H-metric. In that case a boundary solution is the natural optimum.

11.2 The Requested Output Is Unreachable Under the Box

If no feasible input within the box can reproduce r_k exactly, then the bounded solver returns the feasible input that minimizes the output mismatch. The optimum may sit on the boundary because that is where the smallest achievable residual occurs.

11.3 The Objective Has No Interior-Preferring Regularization

The current bounded formulation only minimizes steady-state equation mismatch. It does *not* penalize:

- distance from the current applied input,
- distance from the steady-state anchor,
- distance from the previous bounded target,
- preference for interior points.

Therefore, if the residual is reduced most effectively by moving to a bound, the optimizer has no reason to stay away from that bound.

12 Is the Current Optimization Problem Proper?

12.1 Answer for Feasible Approximation

Yes, if the purpose is:

> "Find the closest feasible steady-state target to the exact unbounded target, measured by the model residual under the input box constraints."

For that purpose, the reduced bounded least-squares problem is a proper convex optimization problem with a global minimizer.

12.2 Answer for Smooth or Interior Targets

Not necessarily, if the purpose is instead:

> "Find a feasible target that also remains smooth, stays near the currently applied input, or avoids saturation whenever possible."

The current formulation does not encode those preferences. So it can be mathematically proper for feasibility restoration while still being too aggressive as a target selector.

13 Alternative Formulations If Boundary Solutions Are Undesirable

If the current bounded solution is too often pinned at limits, the objective can be changed. Common options are:

13.1 Input-Regularized Bounded Least Squares

$$\min_{u_s} \quad \frac{1}{2}\|Gu_s - r_k\|_2^2 + \frac{\rho}{2}\|u_s - u_{ref}\|_2^2 \tag{20}$$

$$\text{subject to} \quad u_{\min} \le u_s \le u_{\max}. \tag{21}$$

Choices for u_{ref} include the current applied input, the previous bounded target, or the steady-state anchor. This often reduces corner-seeking behavior.

13.2 Weighted Regularization

$$\min_{u_s} \quad \frac{1}{2}\|Gu_s - r_k\|_{W_y}^2 + \frac{1}{2}\|u_s - u_{ref}\|_{W_u}^2 \tag{22}$$

$$\text{subject to} \quad u_{\min} \le u_s \le u_{\max}, \tag{23}$$

where $W_y \succeq 0$ and $W_u \succ 0$ encode output priority and input preference.

13.3 Tracking the Applied Input

If the goal is to create an analysis target comparable to the controller action, a term such as

$$\frac{\rho}{2}\|u_s - u_{applied}(k)\|_2^2$$

can make the bounded target much less likely to jump to a far corner of the box.

13.4 Softening Boundary Attraction With Reference Tracking

If the analysis target is intended to answer

> "What feasible target is both model-consistent and close to what the controller is actually doing?"

then the current pure least-squares objective is incomplete. A more appropriate problem is

$$\min_{u_s \in [u_{\min}, u_{\max}]} \quad \frac{1}{2}\|Gu_s - r_k\|_{W_y}^2 + \frac{\rho}{2}\|u_s - u_{ref}\|_{W_u}^2, \tag{24}$$

where u_{ref} could be:

- the current applied input $u_{applied}(k)$,
- the previous bounded target $u_s^{bounded}(k-1)$,
- the nominal steady-state input anchor.

This changes the interpretation from

> "closest feasible point to the exact model target"

to

> "best tradeoff between model fit and target plausibility".

If your goal is to produce a target that should visually align with the controller action, this regularized form is usually more meaningful than plain projection.

14 Practical Interpretation of the Current Sidecar

With the present implementation, the bounded sidecar should be interpreted as answering:

> "Given the current model and frozen disturbance estimate, what is the best feasible steady-state approximation to the exact target if only box constraints are enforced and only equation mismatch is minimized?"

It should *not* be interpreted as:

> "What input target would necessarily be best for a smooth controller-friendly target selection policy?"

15 Bottom-Line Answer

15.1 Are We Using a Proper Optimization Problem?

Yes, for the narrow purpose of feasibility restoration under box constraints. The current bounded least-squares problem is mathematically proper, convex, and globally solvable.

15.2 Is It the Right Problem for an Interpretable or Smooth Target?

Not always. If the bounded target is expected to remain near the controller action, avoid saturation when possible, or stay away from the corners of the admissible box, then the current objective is too weak because it contains no such preference.

15.3 Is the Boundary-Hitting Alone Evidence of a Bug?

No. Given the current run statistics, boundary-hitting is expected. The stronger concern is not solver correctness, but whether the current objective matches the interpretation you want for $u_s^{bounded}$.

16 Per-Step Algorithm Implemented in Code

For each simulation step k:

1. Read $y_{sp}(k)$ and $\hat{d}_k$ from the completed OF-MPC rollout.
2. Form $r_k = y_{sp}(k) - \hat{d}_k$.
3. Compute the exact unbounded steady-state solution.
4. Check whether the exact input target lies inside the box.
5. If the exact target is inside bounds, keep it as the bounded result.
6. Otherwise solve the box-constrained least-squares fallback.
7. Log exact residuals, bounded residuals, bound-activity masks, and exact-versus-bounded differences.

17 Diagnostics

The sidecar stores, for each step:

- exact targets $(x_s^{exact}, u_s^{exact}, y_s^{exact})$
- bound-membership flags and exact bound-violation magnitudes
- bounded targets $(x_s^{bounded}, u_s^{bounded}, y_s^{bounded})$
- bounded residual norms
- lower-bound and upper-bound activity masks
- solve-mode labels such as `exact_bounded` and `exact_unbounded_fallback_bounded_ls`
- exact-versus-bounded differences in both state and input targets

The summary outputs include:

- solve-mode percentages
- exact-within-bounds percentages
- average and maximum exact bound violations
- average and maximum bounded residual norms
- per-input lower and upper bound activity fractions
- event-window tables around setpoint changes and disturbance-estimate jumps

18 Figures

The bounded branch adds the following figures to the existing steady-state debug export:

- exact versus bounded steady-state inputs together with box limits
- applied input versus bounded steady-state input
- exact versus bounded steady-state outputs together with setpoints
- exact residuals and bounded least-squares residuals
- bound-activity time series
- zoomed event-window plots around setpoint changes and large disturbance-estimate updates

19 Scope Boundary

This workflow is only a diagnostic sidecar. It does not modify the offset-free MPC optimization problem, the plant simulation, the observer, or any Lyapunov or safety-filter logic.

References

[1] SciPy documentation, *scipy.optimize.lsq_linear*, https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.optimize.lsq_linear.html.

[2] SciPy documentation, *scipy.optimize.least_squares*, https://docs.scipy.org/doc/scipy-1.13.0/reference/generated/scipy.optimize.least_squares.html.

[3] M. A. Branch, T. F. Coleman, and Y. Li, *A Subspace, Interior, and Conjugate Gradient Method for Large-Scale Bound-Constrained Minimization Problems*, SIAM Journal on Scientific Computing, 21(1), 1–23, 1999.

[4] P. B. Stark and R. L. Parker, *Bounded-Variable Least-Squares: an Algorithm and Applications*, Computational Statistics, 10, 129–141, 1995.